

Sheet (2)

Name : نور إيهاب سمير محمد

ID: 20241063

Implementation Level (Header File) (queue.h)

```
#ifndef QUEUE_H_INCLUDED
#define QUEUE_H_INCLUDED

#define MAX 100

typedef int Type;

typedef struct queue{
    int front;
    int rear;
    Type item[MAX];
} Queue;

void createQueue(Queue *q);
int isEmpty(Queue q);
int isQueueFull(Queue q);
void enqueue(Type item, Queue *q);
Type dequeue(Queue *q);
Type lastElement(Queue q);
Type firstElement(Queue q);
```

```
void destroyQueue(Queue *q);  
void copyQueue(Queue *src, Queue *dest);  
int queueSize(Queue q);  
#endif
```

Implementation Level (queue.c)

```
#include <stdio.h>  
#include "queue.h"  
void createQueue(Queue *q) {  
    q->front = 0;  
    q->rear = MAX - 1;  
}  
int isQueueempty(Queue q) {  
    return (q.front == (q.rear + 1) % MAX);  
}  
int isQueuefull(Queue q) {  
    return (q.front == (q.rear + 2) % MAX);  
}  
void enqueue(Type item, Queue *q) {  
    q->rear = (q->rear + 1) % MAX;  
    q->item[q->rear] = item;  
}  
Type dequeue(Queue *q) {
```

نور إيهاب سمير محمد 20241063

```
Type item = q->item[q->front];  
q->front = (q->front + 1) % MAX;  
return item;  
}
```

Q2

```
Type lastElement(Queue q) {  
    return q.item[q.rear];  
}
```

Q3

```
Type firstElement(Queue q) {  
    return q.item[q.front];  
}
```

Q4

```
void destroyQueue(Queue *q) {  
    q->front = 0;  
    q->rear = MAX - 1;  
}
```

Q5

```
void copyQueue(Queue *src, Queue *dest) {  
    int i;  
    createQueue(dest);  
    int current = src->front;  
    while (current != (src->rear + 1) % MAX) {
```

```
enqueue(src->item[current], dest);  
current = (current + 1) % MAX;  
}  
}
```

Q6

```
int queueSize(Queue q) {  
    int count = 0;  
    int current = q.front;  
    while (current != (q.rear + 1) % MAX) {  
        count++;  
        current = (current + 1) % MAX;  
    }  
    return count;  
}
```

User Level(main.c)

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include "queue.h"  
#define S_MAX 100  
typedef int S_Type;  
typedef struct stack { int top; S_Type item[S_MAX]; } Stack;
```

```
void s_createStack(Stack *s) { s->top = -1; }

int s_isStackempty(Stack s) { return s.top == -1; }

int s_isStackfull(Stack s) { return s.top == S_MAX - 1; }

void s_push(S_Type item, Stack *s) { if (!s_isStackfull(*s)) s->item[++s->top] = item; }

S_Type s_pop(Stack *s) { if (!s_isStackempty(*s)) return s->item[s->top--]; return 0; }

void printQueue(Queue q) {
    if (isQueueempty(q)) {
        printf("Queue is empty.\n");
        return;
    }
    printf("Queue elements (Front -> Rear): ");
    int current = q.front;
    while (current != (q.rear + 1) % MAX) {
        printf("%d ", q.item[current]);
        current = (current + 1) % MAX;
    }
    printf("\n");
}
```

Q12

```
int checkBalance(char exp[]) {
    typedef struct balance_stack { int top; char item[S_MAX]; } BalanceStack;
    BalanceStack s;
```

```
s.top = -1;
for (int i = 0; exp[i] != '\0'; i++) {
    char ch = exp[i];
    if (ch == '(' || ch == '{' || ch == '[') {
        if (s.top == S_MAX - 1) return 0;
        s.item[++s.top] = ch;
    }
    else if (ch == ')' || ch == '}' || ch == ']') {
        if (s.top == -1) return 0;
        char top = s.item[s.top--];
        if ((ch == ')' && top != '(') ||
            (ch == '}' && top != '{') ||
            (ch == ']' && top != '[')) {
            return 0;
        }
    }
}
return s.top == -1;
}
```

Q13

```
void mergeQueues(Queue *sectionQ, Queue *groupQ, Queue *mergedQ) {
    createQueue(mergedQ);

    Type section_code, group_code, merged_code;
```

```
if (queueSize(*sectionQ) != queueSize(*groupQ)) {  
    printf("Error (Q13): Queue sizes do not match. Cannot merge.\n");  
    return;  
}  
  
Queue tempS, tempG;  
copyQueue(sectionQ, &tempS);  
copyQueue(groupQ, &tempG);  
printf("Merging: (Section Code * 10 + Group Code)\n");  
while (!isQueueempty(tempS)) {  
    section_code = dequeue(&tempS);  
    group_code = dequeue(&tempG);  
    merged_code = section_code * 10 + group_code;  
    if (isQueuefull(*mergedQ)) {  
        printf("Error (Q13): Merged Queue Overflow.\n");  
        return;  
    }  
    enqueue(merged_code, mergedQ);  
    printf("-> Merged %d and %d to produce %d\n", section_code, group_code,  
merged_code);  
}  
}
```

Q14

```
void splitGroupIDs(Stack *groupIDStack, Stack *sectionStack, Stack *groupStack) {
```

```
s_createStack(sectionStack);
s_createStack(groupStack);
Stack tempStack;
s_createStack(&tempStack);
S_Type group_id;
while (!s_isStackempty(*groupIdStack)) {
    group_id = s_pop(groupIdStack);
    s_push(group_id, &tempStack);
}
while (!s_isStackempty(tempStack)) {
    group_id = s_pop(&tempStack);
    S_Type section_code = group_id / 10;
    S_Type group_code = group_id % 10;
    if (s_isStackfull(*sectionStack) || s_isStackfull(*groupStack)) {
        printf("Error (Q14): Destination Stack Overflow during splitting.\n");
        return;
    }
    s_push(section_code, sectionStack);
    s_push(group_code, groupStack);
}
s_createStack(groupIdStack);

printf("Group IDs split successfully into Section Codes and Group Codes
stacks.\n");
```



```
}
```

```
int main() {  
    Queue q1, q2;  
    char choice;  
    Type item;  
    createQueue(&q1);  
    Queue sectionQ, groupQ, mergedQ;  
    Stack groupIdStack, sectionStack, groupStack;  
    createQueue(&sectionQ);  
    createQueue(&groupQ);  
    s_createStack(&groupIdStack);  
    enqueue(10, &sectionQ);  
    enqueue(20, &sectionQ);  
    enqueue(1, &groupQ);  
    enqueue(2, &groupQ);  
    s_push(101, &groupIdStack);  
    s_push(202, &groupIdStack);  
    while (1) {  
        printf("\n===== CIRCULAR QUEUE MENU =====\n");  
        printf("(a) Enqueue element (Q1).\n");  
        printf("(b) Dequeue element (Q1).\n");  
        printf("(c) Display first element (Front) - Q8.\n");  
        printf("(d) Display last element (Rear) - Q7.\n");
```

```
printf("(e) Display queue size - Q11.\n");
printf("(f) Copy queue (q1 -> q2) - Q10.\n");
printf("(g) Print queue (q1).\n");
printf("(h) Destroy queue (q1) - Q9.\n");
printf("(i) Check parentheses balance (using Stack) - Q12.\n");
printf("(j) Merge Section & Group Queues - Q13.\n");
printf("(k) Split Group IDs Stack - Q14.\n");
printf("(x) Exit.\n");
printf("Enter choice: ");
scanf(" %c", &choice);
switch (choice) {
    case 'a':
        if (isQueuefull(q1)) {
            printf("Error: Queue Overflow. Cannot enqueue.\n");
        } else {
            printf("Enter element (integer): ");
            scanf("%d", &item);
            enqueue(item, &q1);
            printf("Enqueued: %d\n", item);
        }
        break;
    case 'b':
        if (!isQueueempty(q1)) {
```

```
printf("Dequeued: %d\n", dequeue(&q1));  
} else {  
    printf("Error: Queue Underflow. Queue is empty.\n");  
}  
break;  
case 'c':
```

Q8

```
if (!isQueueempty(q1)) {  
    printf("First element (Front): %d\n", firstElement(q1));  
} else {  
    printf("Error (Q8): Queue is empty.\n");  
}  
break;  
case 'd':
```

Q7

```
if (!isQueueempty(q1)) {  
    printf("Last element (Rear): %d\n", lastElement(q1));  
} else {  
    printf("Error (Q7): Queue is empty.\n");  
}  
break;  
case 'e':
```

Q11

```
printf("Queue size = %d\n", queueSize(q1));
```

```
break;
```

```
case 'f':
```

Q10

```
copyQueue(&q1, &q2);
```

```
printf("Original Queue (q1): "); printQueue(q1);
```

```
printf("Copied Queue (q2): "); printQueue(q2);
```

```
break;
```

```
case 'g':
```

```
printQueue(q1);
```

```
break;
```

```
case 'h':
```

Q9

```
destroyQueue(&q1);
```

```
printf("Queue destroyed (front/rear reset).\n");
```

```
break;
```

```
case 'i': {
```

```
char exp[100];
```

```
printf("Enter expression (e.g., {[()]}): ");
```

```
scanf("%s", exp);
```

```
if (checkBalance(exp)) {
```

```
    printf("Result (Q12): Parentheses are BALANCED.\n");
```

```
} else {
```

```
printf("Result (Q12): Parentheses are NOT balanced.\n");
}
break;
}
case 'j': {
    printf("\n--- Q13: Merging Section & Group Codes ---\n");
    mergeQueues(&sectionQ, &groupQ, &mergedQ);
    printf("Final Merged Queue: "); printQueue(mergedQ);
    break;
}
case 'k': {
    printf("\n--- Q14: Splitting Group IDs Stack ---\n");
    printf("Original Stack content: ");
    Stack tempPrint = groupIdStack;
    while(!s_isStackempty(tempPrint)) { printf("%d ", s_pop(&tempPrint)); }
    printf("\n");
    splitGroupIDs(&groupIdStack, &sectionStack, &groupStack);
    Stack printS = sectionStack, printG = groupStack;
    printf("Section Codes Stack (Top -> Bottom): ");
    while(!s_isStackempty(printS)) { printf("%d ", s_pop(&printS)); }
    printf("\nGroup Codes Stack (Top -> Bottom): ");
    while(!s_isStackempty(printG)) { printf("%d ", s_pop(&printG)); }
```

نور إيهاب سمير محمد 20241063

```
    printf("\n");  
    break;  
}  
case 'x':  
    printf("Exiting program.\n");  
    return 0;  
default:  
    printf("Invalid choice. Please enter a valid option.\n");  
    break;  
}  
}  
}
```